

Лабораторная работа №4

1 Контекстная свобода

Для начала заметим, что атрибут по факту измеряет высоту возможного дерева разбора. Пусть наперед задана строка с n числом a , тогда атрибут лежит от $\lceil \log_2(n+1) \rceil$ до n .

Тогда условие эквивалентно $\lceil \log_2(T_1.attr + 1) \rceil < T_2.attr$, а их атрибуты означают число букв a в них буквально. Отсюда получаем слово для накачки, которое пересекаем с соответствующим регулярным выражением.

Рассмотрим слово $(bba)^{2^n-1}bbaaabb(abb)^{n+1}$.

Понятно, что отдельно компоненты можно накачкой вывести из языка. Но также если и синхронно качать, то выйдем из языка тоже, ведь нулевой накачкой мы значение атрибута у левой части не изменим, так как логарифм берется вверх и такое вычитание не повлияет на атрибут, а правый уже уйдет и нарушится неравенство на атрибуты.

Поэтому язык не является контекстно свободным.

2 Оптимизированный парсер

Рассмотрим идею построения детерминированного алгоритма разбора. Идем от листьев дерева разбора к его вершине.

Первоначально заметим, что подстрока **aaa** может быть получена только по правилу $S \rightarrow aaa$. Поэтому на первом проходе сразу заменяем все вхождения **aaa** на S . По аналогичным причинам сразу же заменяем все вхождения **SbS** на S по правилу $S \rightarrow SbS$.

Далее работаем с третьим правилом $S \rightarrow bbSbb$. Просматриваем строку слева направо. Если встречаем подстроку **bbSbb**, то следует рассмотреть именно это правило. В рамках этого правила будем использовать два указателя: один будет считать возможное число соответствующих букв **a** справа от S , другой — слева от S . Также из-за неравенства справа от S хотя бы одна **a** должна быть, поэтому на правое вхождение такого же S наслоения быть не должно. При этом оба указателя движутся по максимально возможным последовательностям **b**, так как разные вхождения S не могут пересекаться. Если условие из описания языка (соотношение между атрибутами) выполняется, то вся подстрока **bbSbb** заменяется на S ; если условие не выполняется, то данное вхождение пропускается.

После обработки очередного вхождения перемещаемся вправо, начиная именно с самого левого подходящего фрагмента. Это вызвано принципом “жадности”: левая часть неравенства на атрибуты должна быть как можно меньше, чтобы не нарушить возможность разбора последующих частей строки.

Приоритет при замене отдается правилу $S \rightarrow SbS$; только после того, как таких вхождений не осталось, переходим к обработке $S \rightarrow bbSbb$. При этом, как уже сказано, каждый раз выбирается самое левое возможное вхождение.

В конечном счете, если исходная строка принадлежит языку, алгоритм приведет её к единичному символу S .

Таким способом строится детерминированный разбор, который для каждой строки языка выбирает одно конкретное дерево разбора, удовлетворяющее условиям на атрибуты, тем самым реализуя требуемую семантику языка.

3 Оценка сложности оптимизированного парсера

Эффективный парсер имеет кубическую сложность $O(n^3)$ в худшем случае.

Рассмотрим пооперационно:

1. `replaceAll("aaa "S")` — $O(n)$
2. Внешний цикл `while` выполняется не более $O(n)$ раз, так как каждое успешное применение правила сокращает длину строки.
3. `replaceAll("SbS "S")` — $O(n^2)$ в худшем случае (проход по строке и создание новой строки)
4. Внутренний цикл поиска паттернов для правила $S \rightarrow bbSbb$:
 - Поиск подстрок "bbSbb" может выполняться $O(n)$ раз
 - Каждый вызов функции проверки условия (функция `f`) — $O(n)$

Итого для этого этапа: $O(n) \times O(n) = O(n^2)$

Общая оценка:

$$O(n) \times (O(n^2) + O(n^2)) = O(n^3)$$

Таким образом, предложенный алгоритм имеет кубическую временную сложность в худшем случае.

4 Тестирование

Для проверки эффективности предложенного алгоритма было проведено тестирование на двух типах данных:

- словах, принадлежащих языку;
- словах, не принадлежащих языку.

Для каждой длины измерялось время выполнения двух алгоритмов:

- обычного парсера;
- оптимизированного парсера, описанного в разделе 2.

Результаты представлены на графиках ниже. (Значения времени приведены в миллисекундах.)



Рис. 1: Время разбора для слов из языка (обычный парсер)

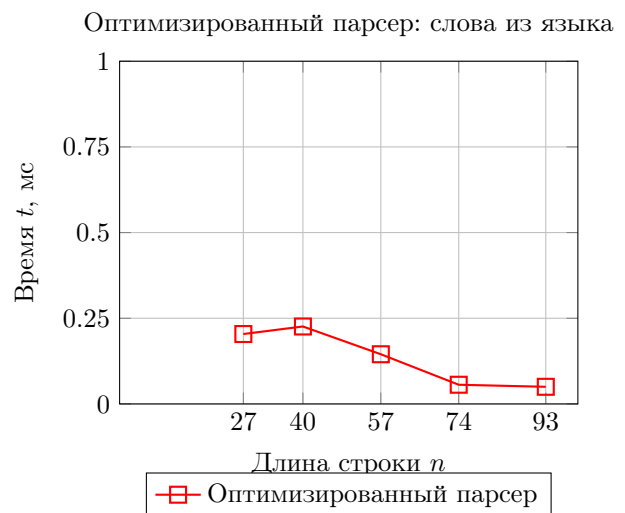


Рис. 2: Время разбора для слов из языка (оптимизированный парсер)



Рис. 3: Время разбора для слов не из языка (обычный парсер)

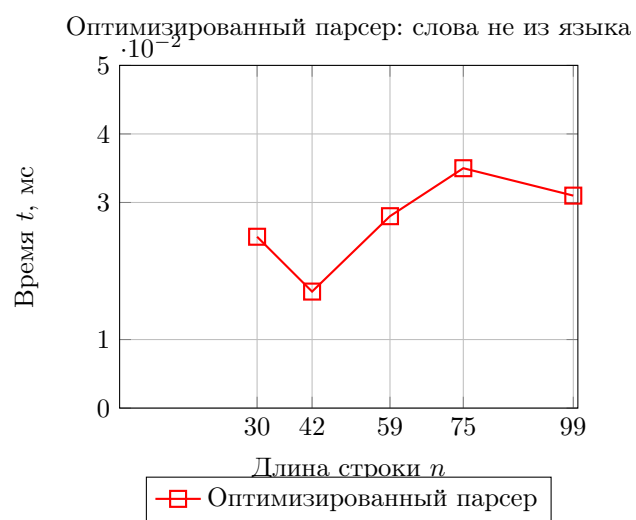


Рис. 4: Время разбора для слов не из языка (оптимизированный парсер)